CITS3402/5507 Assignment 2: a distributed birthday attack with MPI and OpenMP

Shaoming Wu (24914408)

October 2026

1 Algorithm and workload decomposition

A *trial* hashes file A or B with one 64-bit nonce, written as 16 hex digits at byte 16. With n trials per file about $n^2/2^{48}$ cross-file collisions are expected. The K-th collision therefore needs $2^{25}\Gamma(K+\frac{1}{2})/\Gamma(K) \approx 2^{25}\sqrt{K}$ trials (3.0×10^7 for K=1), and every trial must be stored until it is matched.

Process r uses nonces $s\,2^{56} + r\,2^{40} + c$, so trials are never repeated, need no coordination, and every process does equal work. Units of eight nonces alternate between A and B, keeping both populations equal.

The FNV state of the 16-byte prefix is cached; a trial then hashes the nonce and the remaining 64–920 KiB. One FNV chain is limited by multiply latency, so the kernel advances eight nonces over each byte together, giving 7.3× the single-chain throughput. We also derived an exact shortcut. Because $h \oplus c = h + c - 2(h \wedge c)$, and the low byte of an FNV state depends only on the previous low byte, the suffix map is $S(h) = P^n h + C[h \bmod 256] \pmod{2^{64}}$, so a trial costs only 16 steps (`-fast`). As it removes the intended workload, we use it only to expose communication limits.

2 Distributed collision detection

Process $h \bmod p$ owns hash h and stores only its own hashes in an open-addressing table. The table is indexed by a multiplicative hash, so slot positions do not depend on $h \bmod p$. A record that meets a stored record from the other file is a collision, and the slot is then marked. Each hash therefore has one owner and is reported once, so collisions are globally distinct with no extra messages. `toy_hash` ends in a 64-bit finaliser, so ownership is uniform. Owners receive $N/p \pm \sqrt{N/p}$ records, which balances work and memory to within 0.1% at N/p=10^6.

3 Communication, batching and termination

Each process buffers B 16-byte records per destination; records it owns are inserted directly. Full buffers are sent with `MPI_Issend`, at most two in flight per destination. Synchronous sends make this credit-based flow control; with eager `MPI_Isend`, producers flooded slower owners. Owners receive with `MPI_Improbe`/`MPI_Mrecv`, at most p messages per poll, and every waiting loop keeps receiving, which rules out deadlock. The master also polls MPI between its own work units; without asynchronous progress, rendezvous transfers otherwise waited a whole iteration (−25% throughput at B=1024). A larger B means fewer messages but later detection: a buffer fills at rate r/p, delaying candidates by up to Bp/r, which `-flush-ms` (default 100 ms) caps.

To terminate, each process keeps one `MPI_Iallreduce` of its collision count in flight. All processes see the same totals, so they stop at the same reduction once the total reaches K, with no coordinator. They then exchange per-peer message counts (`MPI_Ialltoall`) and drain all in-flight batches. Rank 0 gathers the collisions (`MPI_Gatherv`) and verifies each of the first K on its own, by patching the original files and recomputing `toy_hash`, before writing. Benchmark mode runs exactly N trials through the same pipeline and never stops early.

4 Hybrid MPI+OpenMP

Each process runs one parallel region. While the team hashes chunk i into one of two record buffers (`schedule(dynamic,1)`), the master routes chunk $i-1$, receives and inserts batches, and tests for termination; then it joins the hashing. Lane states are private and the kernel tables are shared read-only. Each unit writes a disjoint slice of the shared record buffer, so no locks are needed. MPI buffers, requests and the table slice belong to the master, and loop control changes only between barriers. Only the master calls MPI, so we request and check `MPI_THREAD_FUNNELED`; `MULTIPLE` would add locking that we do not need.

5 Performance

Platforms. Setonix nodes have two 64-core AMD EPYC 7763 (Cray MPICH). Kaya nodes have two 48-core EPYC 9474F (OpenMPI, `mpirun`). Development used an Apple M4 (8 mixed performance/efficiency cores). From throughput X we report speedup $S = X/X_{\text{serial}}$ and efficiency $E = S/\text{cores}$.

Kernel and time to solution. Per-core throughput falls with file size, from 52 k to 3.7 k trials/s on Setonix and 61 k to 4.0 k on Kaya (Table 1); the fast kernel gives 14–16 M. On four Kaya nodes every pair was solved in 1.7–40 s, against 83–480 s on the laptop.

Table 1: Single-core full-kernel throughput and time to the first verified collision (hybrid; Kaya 4 nodes, 96×4; M4 2×4).

		full kernel, trials/s per core			time to collision t_1 (s)	
pair	KiB	Setonix	Kaya	Apple M4	Kaya (384 c.)	Apple M4 (8 c.)
1_alpha	65	52.2 k	61.1 k	113.4 k	1.7	82.9
2_beta	130	26.3 k	30.4 k	57.0 k	2.0	87.5
3_gamma	230	14.8 k	16.5 k	32.2 k	3.9	128.8
4_delta	350	9.8 k	11.2 k	21.2 k	12.7	273.8
5_epsilon	500	6.8 k	7.5 k	14.7 k	8.8	140.8
6_zeta	920	3.7 k	4.0 k	8.0 k	39.8	480.3

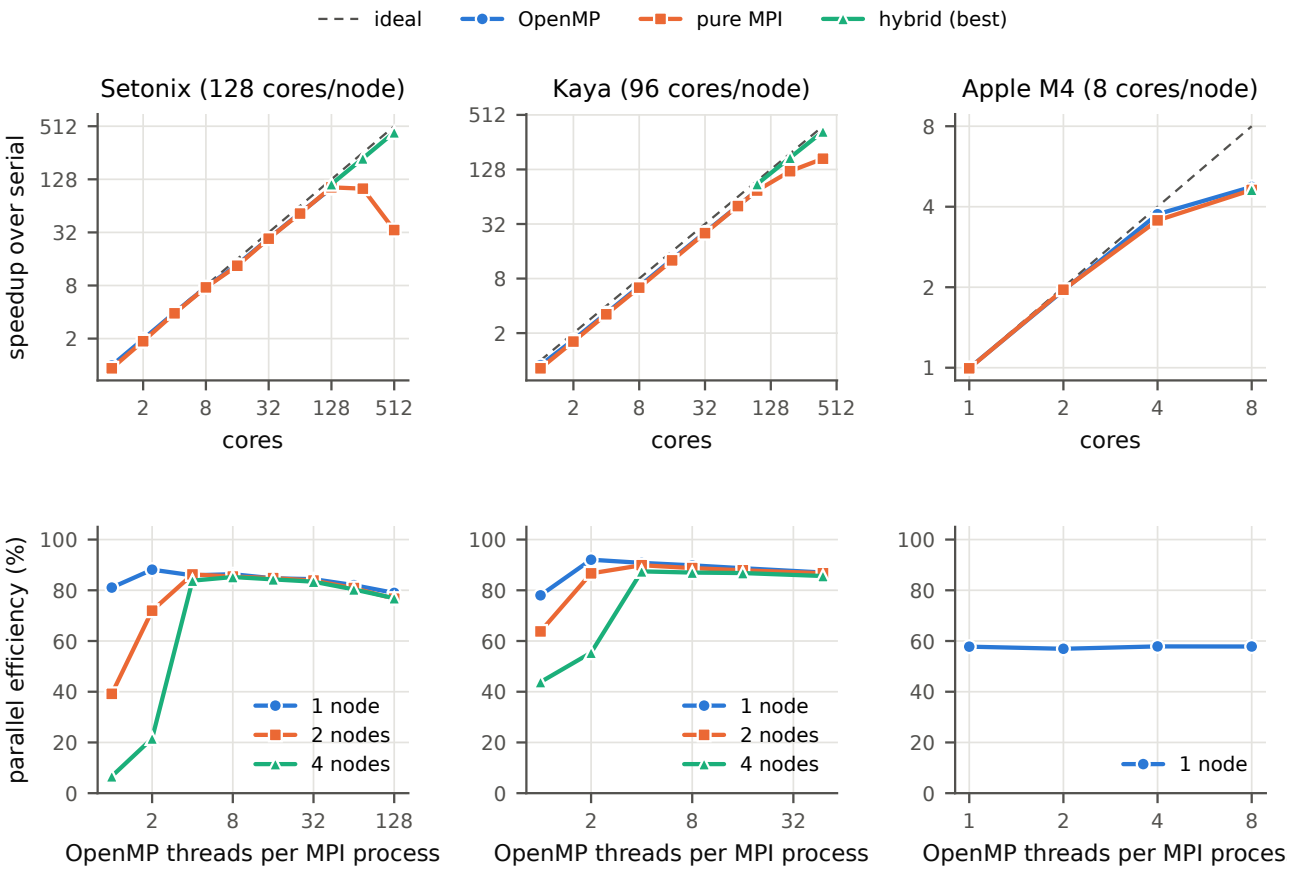


Figure 1: Fixed-work scaling (gamma). Top: best speedup per core count. Bottom: whole-node efficiency against threads per process (1 = pure MPI).

Scaling. On one cluster node all programs reach $E = 78$–81%, and the best hybrid split reaches 88–92% (Figure 1, Table 2). OpenMP loses as much, so this is probably the clock dropping from single-core boost under full load. Across nodes, pure MPI falls to 39% and 7% on two and four Setonix nodes (64% and 44% on Kaya). Hybrid with 4–8 threads per process keeps 84–90% (S=436 on 512 cores). The cause is the timed flush: each process sends partial batches to all $p-1$ owners every 100 ms, so the message count grows as p^2. With 512 processes, 25 M messages carried 2.6 records each. Using t threads per process divides p by t and multiplies records per message by about t^2 (161 for 64×8).

Batch size. On 384 Kaya processes the full kernel is within 4% of its best from B=16, while the fast kernel gains 19× from B=1 to B=4096 (Figure 2). On Setonix, B=1 with 512 processes took 796 s, and its fast-kernel run was killed for running out of memory. Larger batches delay detection by about Bp/r. With B=4096 the first collision took 32 s and 582 M trials (35 s predicted), but 2.3 s with the flush.

Multiple collisions. Mean trials are within 5% of $2^{25}\Gamma(K+\frac{1}{2})/\Gamma(K)$ for $K \geq 2$ (Figure 3), so K=16 costs 3.4–4.0× the time of K=1, not 16×: each stored record can pair with every later one. Termination adds no messages but has latency. With the fast kernel on 384 processes, the 30 M trials take 0.1 s, but detection and the global stop took 1.5–2 s, so 200–540 collisions were found for K=1.

Trade-offs. A trial hashes 64–920 KiB but sends 16 bytes, so computation dominates while messages stay large. At scale the limits are message count ($\propto p^2$), detection delay (Bp/r) and buffer memory ($16pB$ bytes per process). Hybrid reduces all three, so for multi-node runs we choose 4–8 threads per process, B of 256–1024 and the 100 ms flush. Node-level aggregation, or a flush interval that grows with p, would help pure MPI.

Table 2: Fixed-work results for whole nodes (best hybrid split).

program	nodes	ranks×thr.	cores	X (trials/s)	S	E
Setonix						
serial	1	1×1	1	14.8 k	1.0	100%
OpenMP	1	1×128	128	1.51 M	101.7	79%
MPI	1	128×1	128	1.54 M	103.8	81%
hybrid	1	64×2	128	1.67 M	112.8	88%
MPI	2	256×1	256	1.49 M	100.3	39%
hybrid	2	64×4	256	3.28 M	220.8	86%
MPI	4	512×1	512	505.4 k	34.0	7%
hybrid	4	64×8	512	6.48 M	436.4	85%
Kaya						
serial	1	1×1	1	17.3 k	1.0	100%
OpenMP	1	1×96	96	1.30 M	75.1	78%
MPI	1	96×1	96	1.30 M	74.9	78%
hybrid	1	48×2	96	1.53 M	88.4	92%
MPI	2	192×1	192	2.12 M	122.5	64%
hybrid	2	48×4	192	2.98 M	172.6	90%
MPI	4	384×1	384	2.91 M	168.3	44%
hybrid	4	96×4	384	5.81 M	335.8	87%
Apple M4						
serial	1	1×1	1	32.4 k	1.0	100%
OpenMP	1	1×8	8	153.6 k	4.7	59%
MPI	1	8×1	8	149.6 k	4.6	58%
hybrid	1	2×4	8	149.8 k	4.6	58%

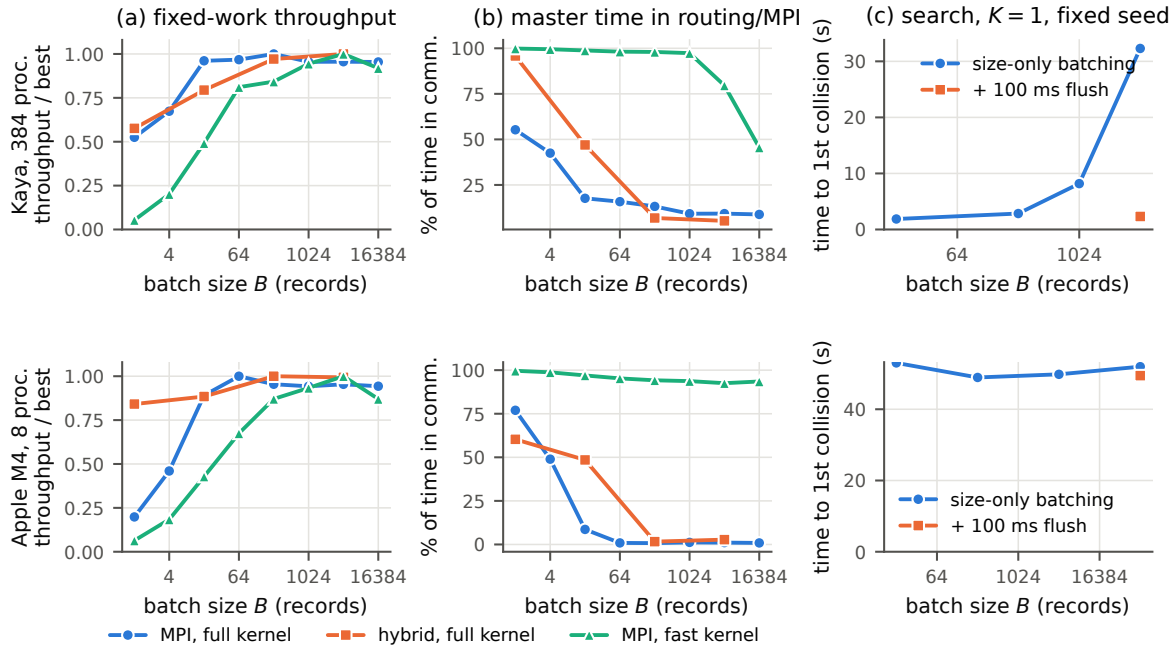


Figure 2: Batch size B (alpha, no timed flush unless stated): (a) throughput; (b) master time spent communicating, overlapped with hashing in the hybrid; (c) time to the first collision.

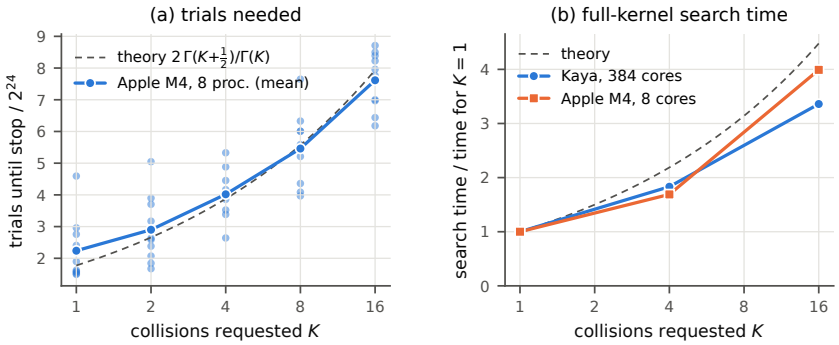


Figure 3: K distinct collisions (alpha): (a) trials for ten seeds (dots) and their mean against theory; (b) full-kernel time relative to K=1.